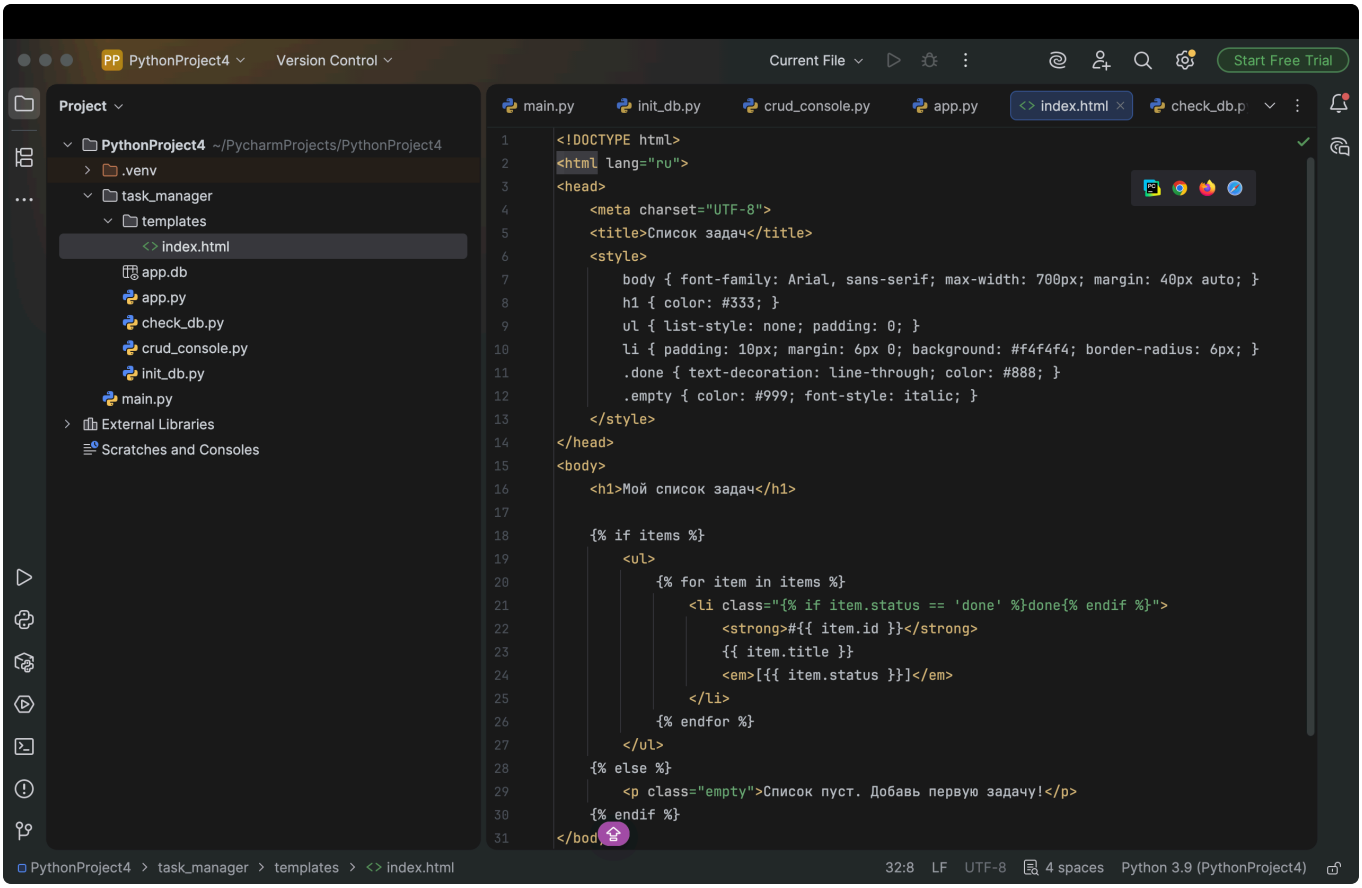
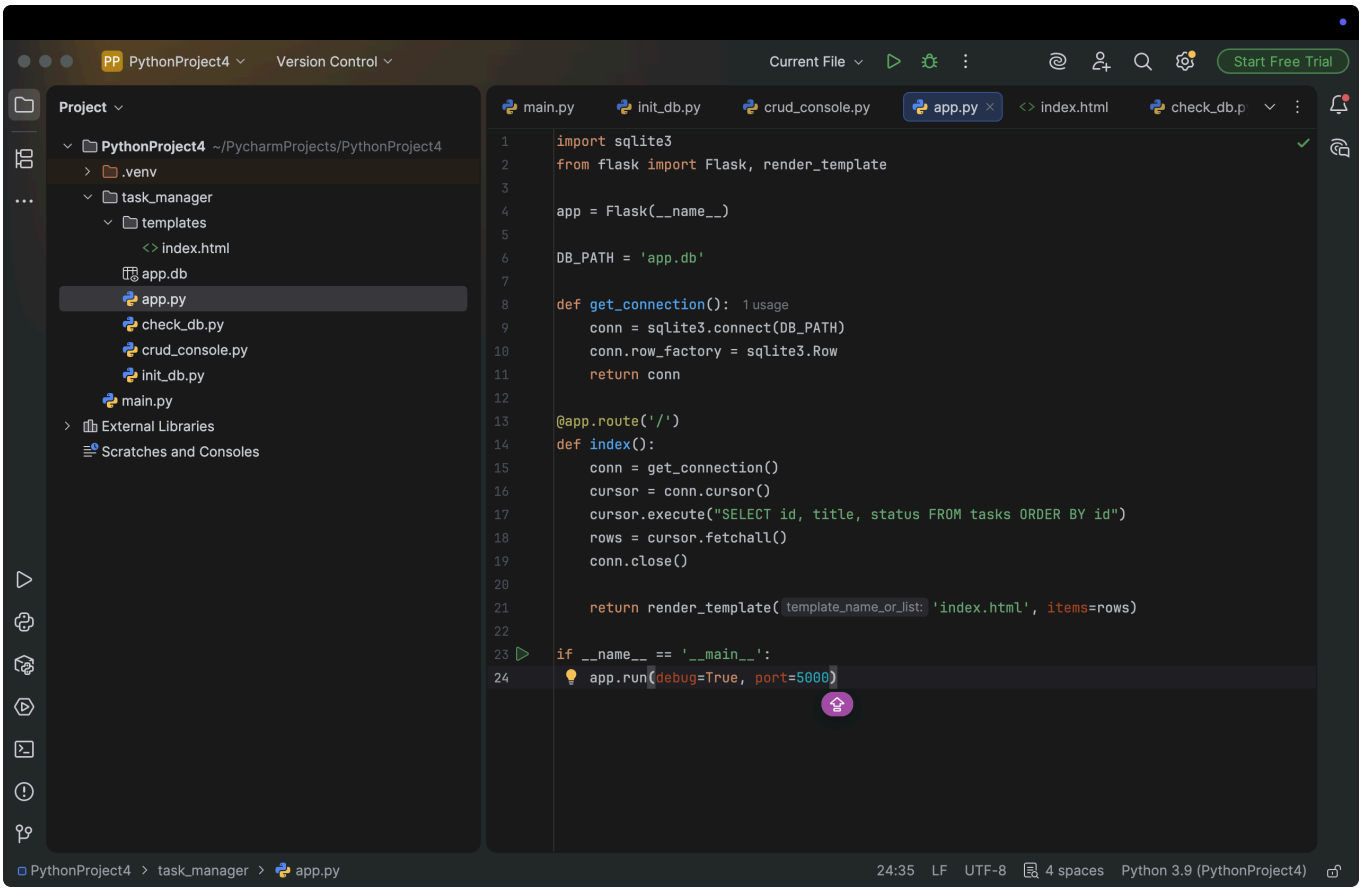
Интеграция Flask и SQLite (Чтение данных)

Ястребкова Лилия ЗИСП11-44

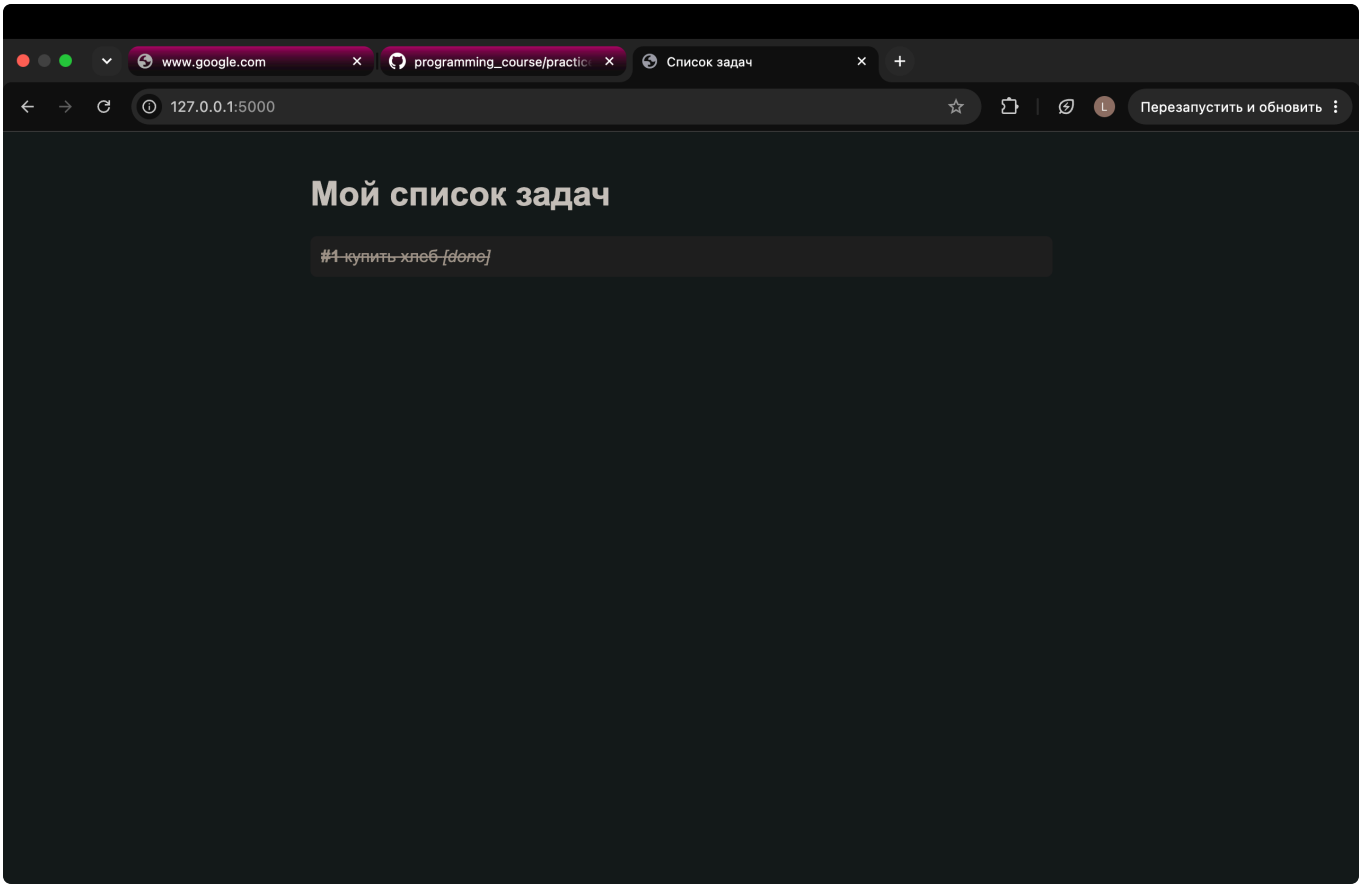
Шаг 2. Создание шаблона `templates/index.html`



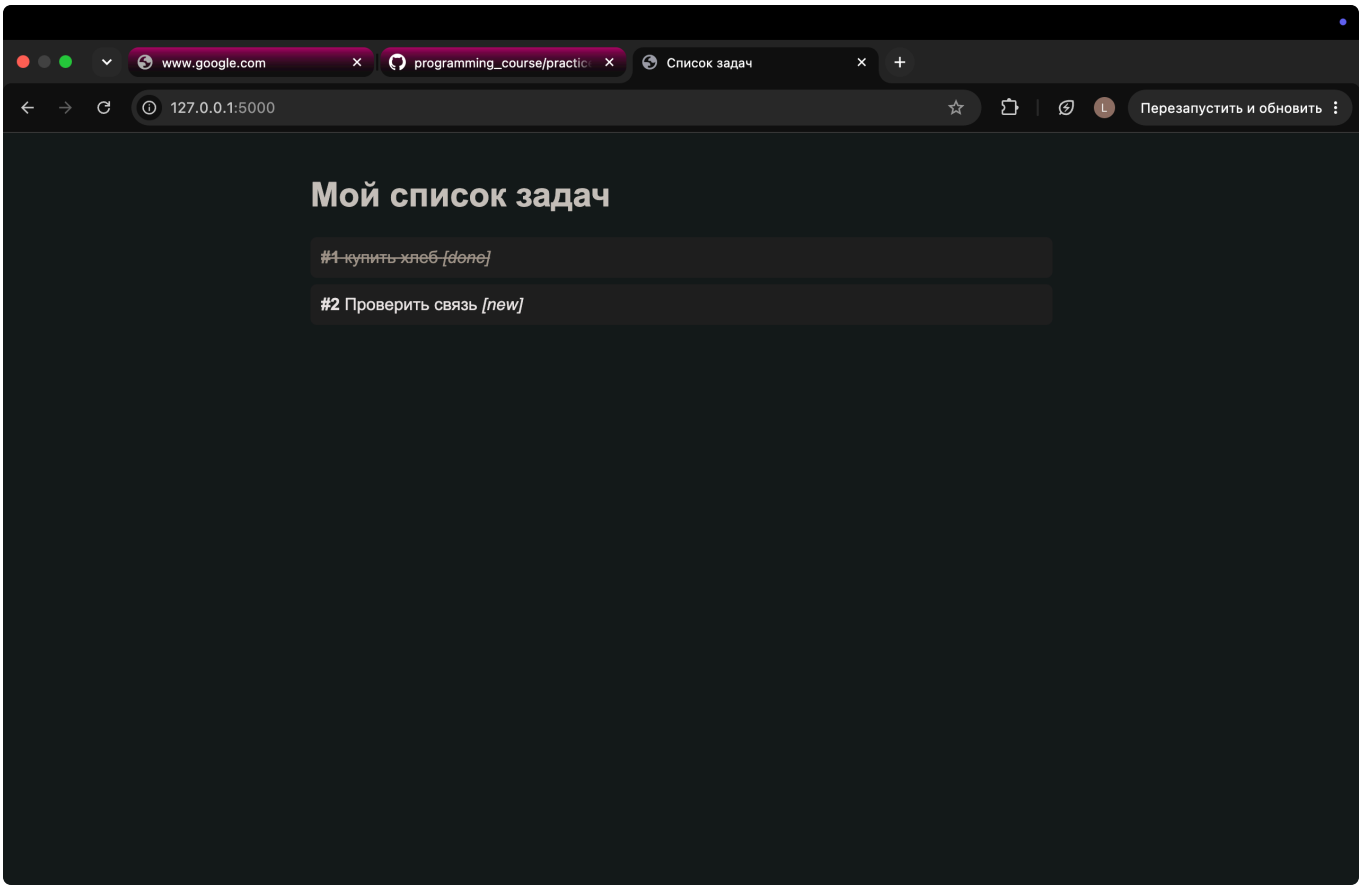
Шаг 3. Переписываем `app.py`: чтение из базы

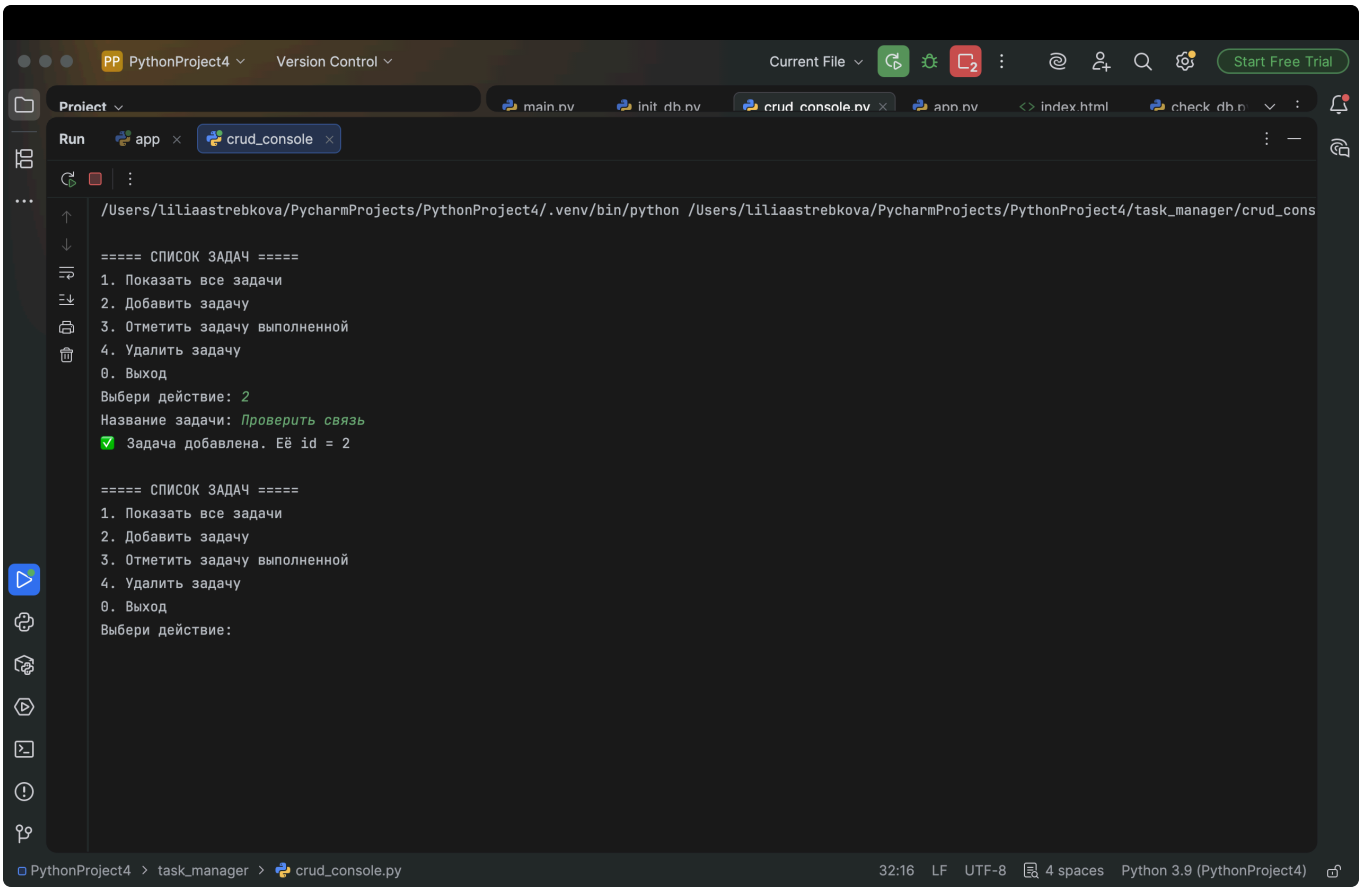


Шаг 4. Запуск и проверка

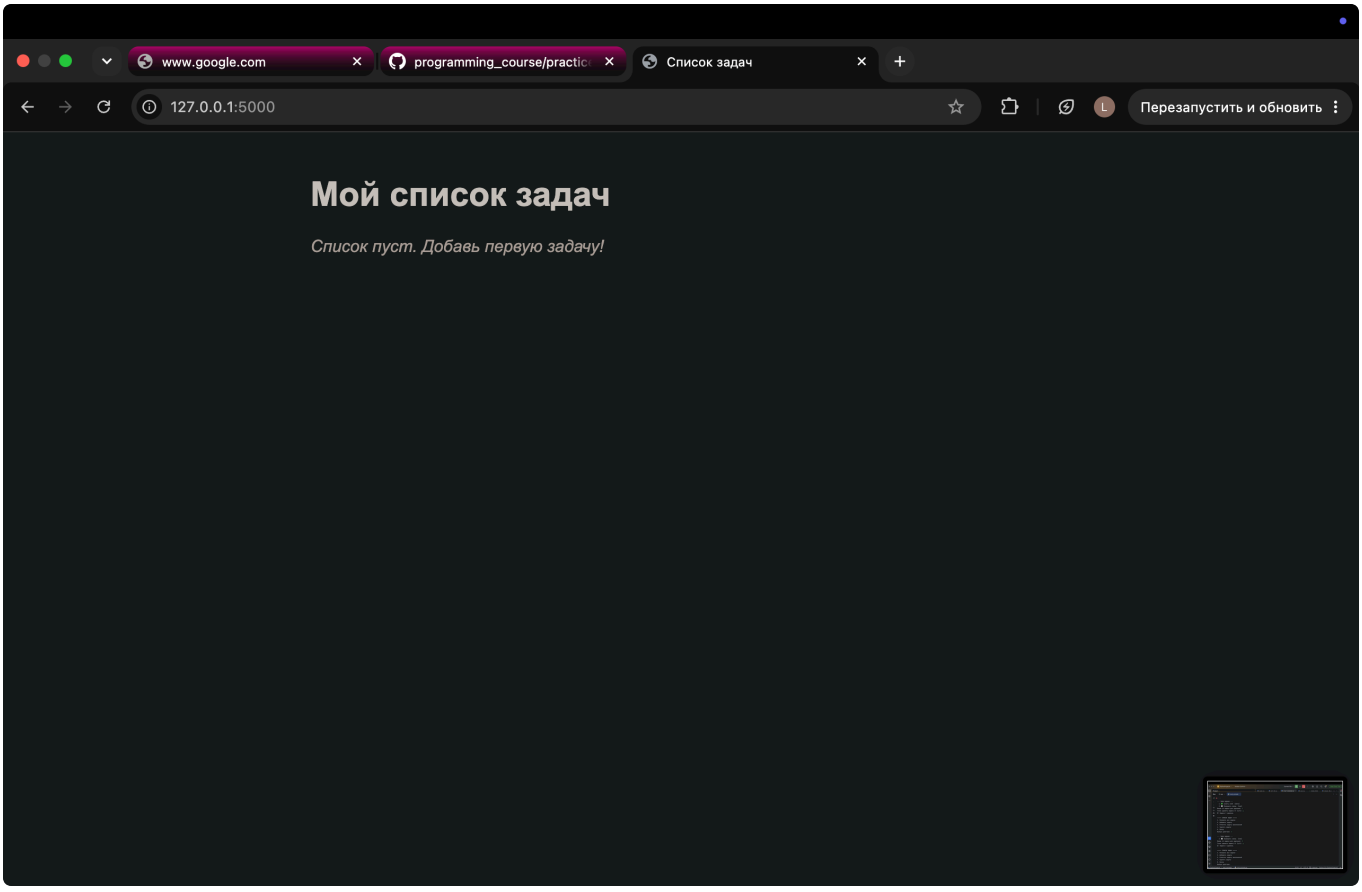


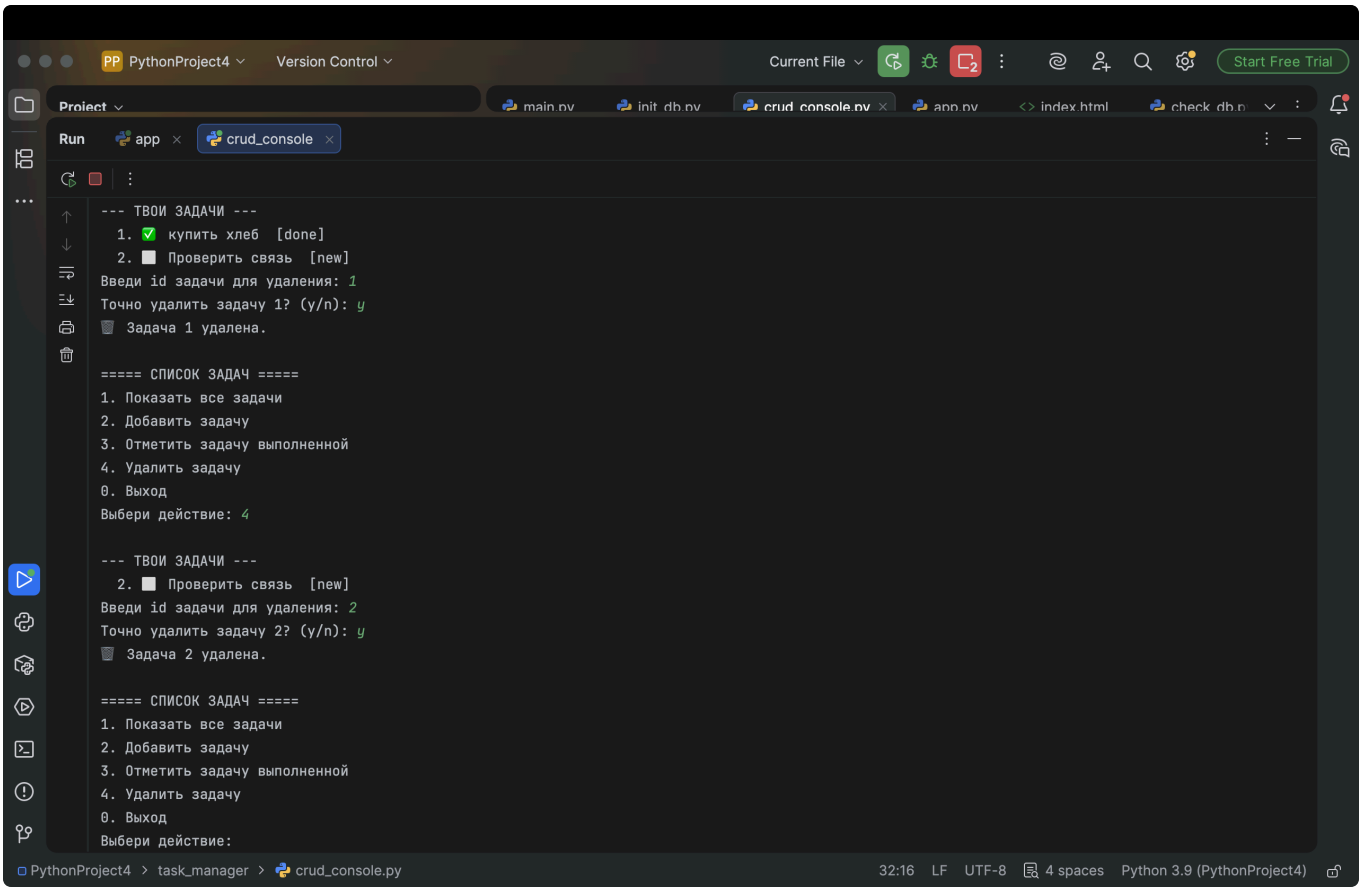
Шаг 5. Проверка «живой» связи с базой



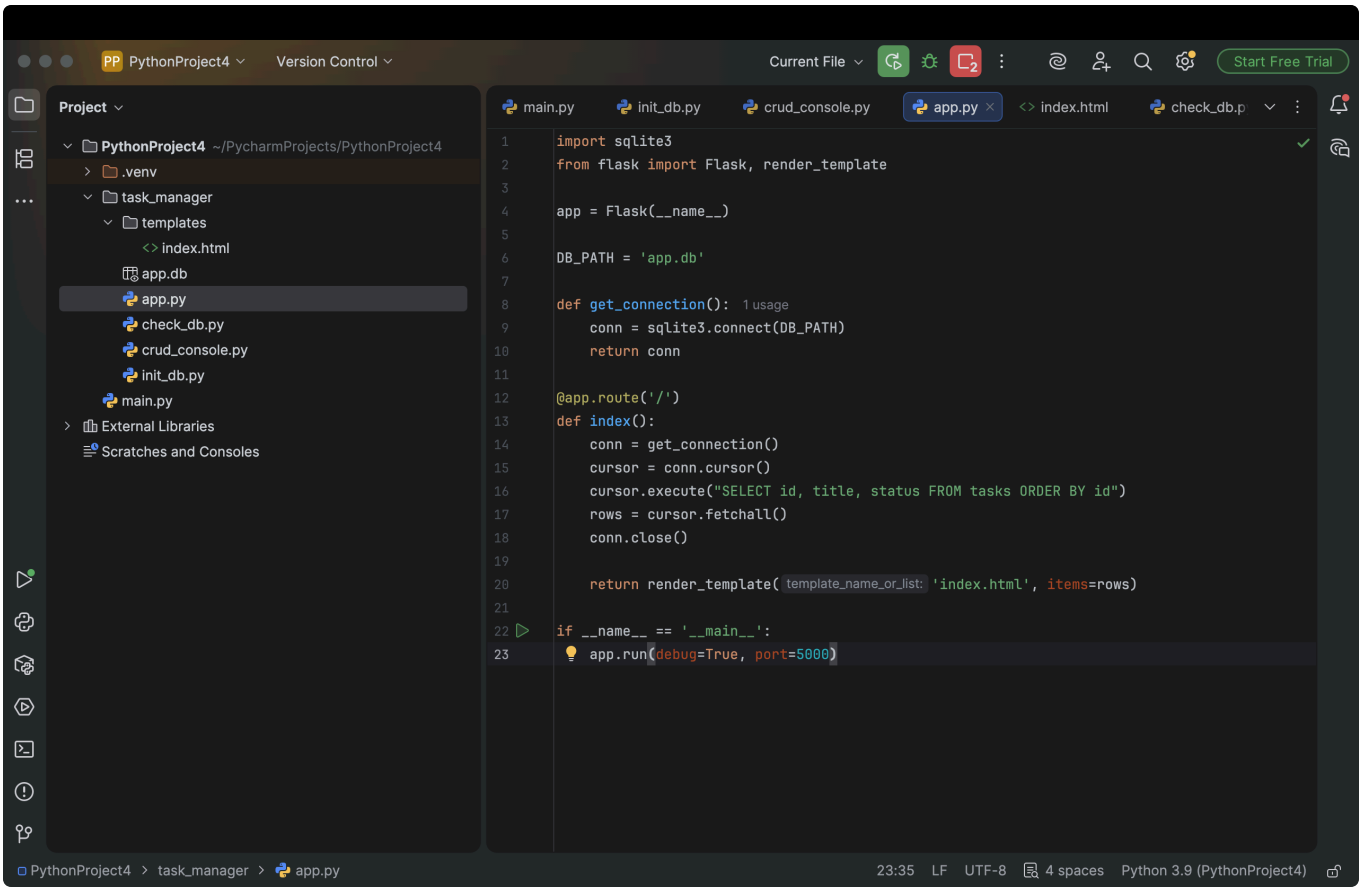


Шаг 6. Проверка пустой базы





Шаг 7. Эксперимент: `sqlite3.Row` vs обычный кортеж



1. Зачем нужен шаблонизатор Jinja2, если можно вернуть HTML-строку из функции?

Возврат HTML строкой из Python-кода работает, но плохо масштабируется: код и разметка перемешиваются, страницу сложно читать и править, редактор не подсвечивает HTML внутри строки Python, а повторяющиеся куски (шапка, меню) невозможно переиспользовать. Jinja2 разделяет ответственность: HTML лежит отдельным файлом, Python только передаёт в него данные. Плюс шаблонизатор умеет циклы и условия (`{% for %}` , `{% if %}`) - на строках в коде это превращалось бы в неудобную склейку фрагментов. Есть и важный бонус безопасности: `{{ ... }}` автоматически экранирует HTML, защищая от XSS-инъекций.

2. Чем `{{ ... }}` отличается от `{% ... %}` в Jinja2?

`{{ ... }}` - вывод значения в HTML: подставляет содержимое переменной или выражения (например, `{{ item.title }}`). `{% ... %}` - управляющая конструкция: циклы, условия, блоки (`{% for item in items %}` , `{% if item.status == 'done' %}` , `{% endfor %}`). Сама по себе она ничего не выводит, а лишь управляет логикой рендеринга. Есть ещё `{# ... #}` - комментарий, который вообще не попадает в итоговый HTML.

3. Что делает `render_template('index.html', items=rows)` ?

Она делает две вещи: находит файл `templates/index.html` и прогоняет его через Jinja2, подставляя переданные именованные аргументы как переменные шаблона (здесь `items=rows` - в шаблоне доступна переменная `items` с данными из базы). Возвращается готовая HTML-страница, которую Flask отправляет браузеру.

4. Почему при чтении данных не нужен `conn.commit()`?

`commit()` фиксирует изменения на диске, а `SELECT` ничего не меняет - он только читает строки и возвращает их программе. Фиксировать нечего, поэтому в цепочке «подключение → SELECT → fetchall → close» commit отсутствует. Он нужен только после операций, изменяющих данные: `INSERT`, `UPDATE`, `DELETE`.

5. Что такое `sqlite3.Row` и почему `item.title` работает только с ним?

`sqlite3.Row` - устанавливается через `conn.row_factory = sqlite3.Row` после подключения. По умолчанию `fetchall()` возвращает кортежи, к которым обращаются только по индексу: `row[0]`, `row[1]`. Атрибутов и имён колонок у кортежа нет, поэтому в шаблоне `{{ item.title }}` для кортежа приведёт к `UndefinedError`. После установки `row_factory` каждая строка становится объектом, поддерживающим доступ по имени колонки: `row['title']`, и в шаблоне запись `{{ item.title }}` начинает работать. Это удобнее и надёжнее - код не ломается при изменении порядка колонок в SELECT.

6. Куда Flask кладёт файлы шаблонов по умолчанию и можно ли это изменить?

По умолчанию Flask ищет шаблоны в папке `templates` рядом с файлом приложения - это соглашение задаётся при создании `Flask(__name__)`: имя модуля используется, чтобы определить расположение проекта. Изменить можно: у конструктора Flask есть параметр `template_folder`, например `Flask(__name__, template_folder='my_html')`, но в стандартных проектах так не делают и оставляют имя `templates`.

7. Что выведет шаблон, если `items` - пустой список?

Сработает ветка `{% else %}` условия `{% if items %}`: пустой список считается ложным в Jinja2 (как в Python), поэтому цикл не выполнится ни разу, а пользователь увидит сообщение «Список пуст. Добавь первую задачу!». Именно для этого случая в шаблоне и сделана проверка.